Complete Project Documentation: Retail Data Warehouse to OLTP/ROLAP/MOLAP Migration

Project Overview

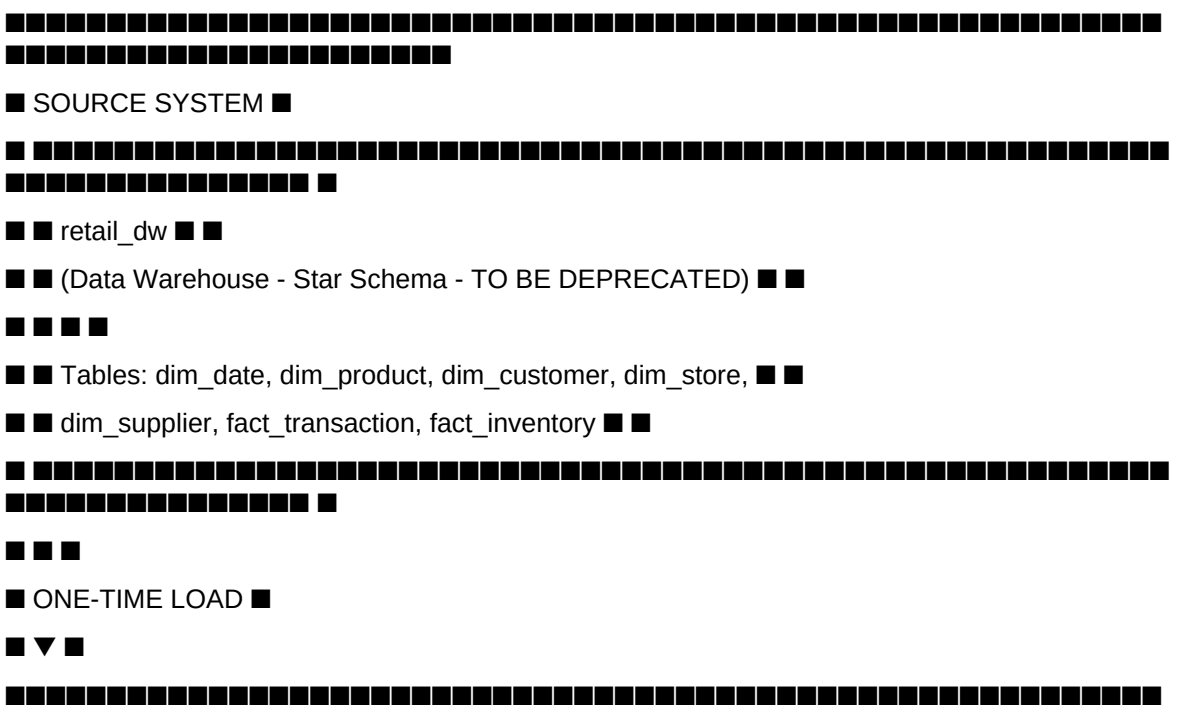
This document provides comprehensive documentation for the migration and transformation of a retail data warehouse (`retail_dw`) into a complete three-tier analytical system comprising OLTP (Transaction Processing), ROLAP (Relational OLAP), and MOLAP (Multidimensional OLAP) databases.

Table of Contents

---

1. Project Architecture

1.1 High-Level Architecture Diagram



■ TRANSACTIONAL SYSTEM ■

■ ■■
■■■■■■■■■■■■ ■

■ ■ retail_oltp ■ ■

■ ■ (OLTP - 3NF Normalized - For Transaction Processing) ■ ■

■ ■ ■ ■

■ ■ Tables: customers, products, stores, suppliers, sales_orders, ■ ■

■ ■ sales_order_items, inventory, categories, brands ■ ■

■ ■■
■■■■■■■■■■■■ ■

■ ■ ■

■ ETL PROCESS ■

■ ▼ ■

■■■
■■■■■■■■■■■■■■■■■

■ ANALYTICAL SYSTEMS ■

■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■ ■

■ ■ retail_rolap ■ ■ retail_molap ■ ■

■ ■ (ROLAP - Star Schema) ■ ■ (MOLAP - Cube System) ■ ■

■ ■ ■ ■ ■ ■

■ ■ - dim_date ■ ■ - cube_sales_* ■ ■

■ ■ - dim_product ■ ■ - cube_product_hierarchy ■ ■

■ ■ - dim_store ■ ■ - cube_store_hierarchy ■ ■

■ ■ - dim_customer ■ ■ - cube_time_series ■ ■

■ ■ - dim_supplier ■ ■ - cube_customer_segmentation■ ■

■ ■ - fact_sales ■ ■ ■ ■

■ ■ - agg_* (pre-aggregated) ■ ■ ■ ■

■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■ ■

■■■
■■■■■■■■■■■■■■■■■

text

1.2 Technology Stack

Component	Technology	Version
Database	MySQL Community Server	8.0.41
Source Data	retail_dw (existing DW)	-
Target OLTP	retail_oltp	New Database

| Target ROLAP | retail_rolap | New Database |

| Target MOLAP | retail_molap | New Database |

| Language | SQL (MySQL Native) | - |

1.3 Data Flow Overview

retail_dw ■■(One-Time Load)■■■ retail_oltp ■■(ETL)■■■ retail_rolap ■■(ETL)■■■ retail_molap

■ ■ ■ ■

■ ■ ■ ■

▼ ▼ ▼ ▼

Deprecated Transaction Processing Relational Analytics Cube Analytics

text

1.4 System Characteristics

| System | Schema Type | Purpose | Update Frequency | Query Type |

|--------|-------------|---------|------------------|------------|

| retail_dw | Star Schema | Historical reporting | Deprecated | SELECT only |

| retail_oltp | 3NF Normalized | Transaction processing | INSERT/UPDATE/DELETE | OLTP (many writes) |

| retail_rolap | Star Schema | Relational analytics | Batch ETL | OLAP (complex queries) |

| retail_molap | Cube structures | Multidimensional analysis | Batch ETL | Pre-aggregated queries |

---

2. Source System: retail_dw

2.1 Overview

`retail_dw` is the **source data warehouse** that contains historical retail data. After migration, this database will be **deprecated** and all operations will move to `retail_oltp`.

2.2 Schema Structure

```sql

retail_dw/

■■■ dim_date -- Date dimension

■■■ dim_product -- Product dimension

■■■ dim_customer -- Customer dimension

■■■ dim_store -- Store dimension

■■■ dim_supplier -- Supplier dimension

■■■ fact_transaction -- Sales fact table

■■■ fact_inventory -- Inventory fact table

2.3 Table Details

dim_date
```

Column Type Description

date_id INT Primary key (YYYYMMDD)

full_date DATE Actual date

year INT Year

quarter INT Quarter (1-4)

month INT Month (1-12)

month_name VARCHAR(20) Month name

day INT Day of month

day_of_week INT Day of week (1-7)

weekday_name VARCHAR(20) Day name

is_weekend VARCHAR(10) Weekend flag

dim_product

Column Type Description

product_id VARCHAR(20) Primary key

product_name VARCHAR(255) Product name

brand VARCHAR(100) Brand name

category VARCHAR(100) Category

sub_category VARCHAR(100) Sub-category

mrp DECIMAL(10,2) Maximum retail price

cost_price DECIMAL(10,2) Cost price

gross_margin_pct DECIMAL(10,4) Gross margin %

is_perishable VARCHAR(10) Perishable flag

dim_customer

Column Type Description

customer_id VARCHAR(20) Primary key

customer_name VARCHAR(255) Customer name

gender VARCHAR(20) Gender

age_group VARCHAR(50) Age group

city VARCHAR(100) City

state VARCHAR(100) State

city_tier INT Tier (1,2,3)

is_registered VARCHAR(10) Registered flag

loyalty_member VARCHAR(10) Loyalty flag

avg_monthly_spend_inr DECIMAL(12,2) Average monthly spend

dim_store

Column Type Description

store_id VARCHAR(20) Primary key

store_name VARCHAR(255) Store name

city VARCHAR(100) City
state VARCHAR(100) State
city_tier INT Tier
cluster_zone VARCHAR(50) Cluster zone
store_type VARCHAR(50) Store type
fact_transaction
Column Type Description
transaction_key BIGINT Primary key
transaction_id VARCHAR(30) Transaction ID
bill_date DATE Transaction date
store_id VARCHAR(20) FK to dim_store
customer_id VARCHAR(20) FK to dim_customer
product_id VARCHAR(20) FK to dim_product
quantity INT Quantity sold
mrp DECIMAL(10,2) MRP
discount_pct DECIMAL(8,4) Discount %
discount_amount DECIMAL(10,2) Discount amount
total_sale_amount DECIMAL(14,2) Total sale amount
payment_mode VARCHAR(50) Payment method
2.4 Key Characteristics
Characteristic Description
Schema Type Star Schema (denormalized dimensions)
Purpose Historical reporting and analytics
Grain Transaction line item (fact_transaction)
Date Range Historical data (up to Dec 30, 2025)
Status To be deprecated after migration
2.5 Sample Data
sql
-- dim_product sample
product_id: 'PROD001'
product_name: 'iPhone 14'
brand: 'Apple'
category: 'Electronics'
mrp: 79900.00
cost_price: 65000.00

-- fact_transaction sample
transaction_id: 'TXN12345'
bill_date: 2025-12-30

store_id: 'ST001'

customer_id: 'CUST001'

product_id: 'PROD001'

quantity: 1

total_sale_amount: 79900.00

3. Target System: retail_oltp

3.1 Overview

retail_oltp is the transaction processing system designed for high-volume insert, update, and delete operations. It uses a 3NF (Third Normal Form) design to minimize data redundancy and ensure data integrity.

3.2 Schema Structure

text

retail_oltp/

■■■ Lookup Tables/

■ ■■■ lookup_gender -- M, F, O

■ ■■■ lookup_payment_mode -- Cash, Credit Card, UPI, etc.

■ ■■■ lookup_order_status -- Pending, Completed, Cancelled, etc.

■ ■■■ lookup_channel -- Store, Online, App, Call Center

■

■■■ Master Tables/

■ ■■■ categories -- Product category hierarchy

■ ■■■ brands -- Product brands

■ ■■■ suppliers -- Supplier master data

■ ■■■ stores -- Store master data

■ ■■■ customers -- Customer master data

■ ■■■ products -- Product master data

■

■■■ Relationship Tables/

■ ■■■ product_suppliers -- Many-to-many product-supplier mapping

■ ■■■ inventory -- Current stock levels

■

■■■ Transaction Tables/

■■■ sales_orders -- Order header

■■■ sales_order_items -- Order line items

3.3 Table Definitions

Lookup Tables

lookup_gender

sql

CREATE TABLE lookup_gender (
gender_code CHAR(1) PRIMARY KEY,
gender_name VARCHAR(20)
);
-- Data: ('M', 'Male'), ('F', 'Female'), ('O', 'Other')
lookup_payment_mode

sql
CREATE TABLE lookup_payment_mode (
payment_mode_id TINYINT PRIMARY KEY AUTO_INCREMENT,
payment_mode_name VARCHAR(50) UNIQUE
);
-- Data: Cash, Credit Card, Debit Card, UPI, Net Banking, Wallet
lookup_order_status

sql
CREATE TABLE lookup_order_status (
status_id TINYINT PRIMARY KEY AUTO_INCREMENT,
status_name VARCHAR(30) UNIQUE
);
-- Data: Pending, Processing, Completed, Cancelled, Returned
lookup_channel

sql
CREATE TABLE lookup_channel (
channel_id TINYINT PRIMARY KEY AUTO_INCREMENT,
channel_name VARCHAR(50) UNIQUE
);
-- Data: Store, Online, App, Call Center
Master Tables
categories (Self-referential hierarchy)

sql
CREATE TABLE categories (
category_id INT PRIMARY KEY AUTO_INCREMENT,
category_name VARCHAR(100) NOT NULL,
parent_category_id INT,
level TINYINT DEFAULT 1,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (parent_category_id) REFERENCES categories(category_id)
);

brands

sql

```
CREATE TABLE brands (
brand_id INT PRIMARY KEY AUTO_INCREMENT,
brand_name VARCHAR(100) NOT NULL UNIQUE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

suppliers

sql

```
CREATE TABLE suppliers (
supplier_id INT PRIMARY KEY AUTO_INCREMENT,
supplier_code VARCHAR(20) NOT NULL UNIQUE,
supplier_name VARCHAR(255) NOT NULL,
supplier_type VARCHAR(100),
city VARCHAR(100),
state VARCHAR(100),
payment_days INT DEFAULT 30,
lead_time_days INT DEFAULT 7,
annual_contract_value_lakh DECIMAL(12,2),
is_preferred_vendor BOOLEAN DEFAULT FALSE,
years_associated INT DEFAULT 0,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

stores

sql

```
CREATE TABLE stores (
store_id INT PRIMARY KEY AUTO_INCREMENT,
store_code VARCHAR(20) NOT NULL UNIQUE,
store_name VARCHAR(255) NOT NULL,
city VARCHAR(100),
state VARCHAR(100),
city_tier TINYINT,
cluster_zone VARCHAR(50),
store_type VARCHAR(50),
store_size_sqft INT,
opening_date DATE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

);

customers

sql

```
CREATE TABLE customers (
customer_id INT PRIMARY KEY AUTO_INCREMENT,
customer_code VARCHAR(20) NOT NULL UNIQUE,
customer_name VARCHAR(255) NOT NULL,
email VARCHAR(100),
phone VARCHAR(20),
gender CHAR(1),
age_group VARCHAR(50),
city VARCHAR(100),
state VARCHAR(100),
city_tier INT,
is_registered BOOLEAN DEFAULT FALSE,
loyalty_member BOOLEAN DEFAULT FALSE,
avg_monthly_spend_inr DECIMAL(12,2),
visit_frequency_per_month INT,
registration_date DATE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);
```

products

sql

```
CREATE TABLE products (
product_id INT PRIMARY KEY AUTO_INCREMENT,
product_code VARCHAR(20) NOT NULL UNIQUE,
product_name VARCHAR(255) NOT NULL,
brand_id INT,
category_id INT,
sub_category_id INT,
mrp DECIMAL(10,2) NOT NULL,
cost_price DECIMAL(10,2) NOT NULL,
gross_margin_pct DECIMAL(10,4) GENERATED ALWAYS AS (
((mrp - cost_price) / NULLIF(mrp, 0)) * 100
) STORED,
is_perishable BOOLEAN DEFAULT FALSE,
```

```
veg_nonveg VARCHAR(20),
max_qty_per_transaction INT DEFAULT 10,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (brand_id) REFERENCES brands(brand_id),
FOREIGN KEY (category_id) REFERENCES categories(category_id),
FOREIGN KEY (sub_category_id) REFERENCES categories(category_id)
);
```

Transaction Tables

sales_orders

```
sql
CREATE TABLE sales_orders (
order_id BIGINT PRIMARY KEY AUTO_INCREMENT,
order_number VARCHAR(30) NOT NULL UNIQUE,
customer_id INT,
store_id INT NOT NULL,
order_date DATETIME NOT NULL,
status_id TINYINT DEFAULT 1,
payment_mode_id TINYINT,
channel_id TINYINT DEFAULT 1,
total_amount DECIMAL(12,2),
discount_amount DECIMAL(12,2) DEFAULT 0,
net_amount DECIMAL(12,2),
is_return BOOLEAN DEFAULT FALSE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
FOREIGN KEY (store_id) REFERENCES stores(store_id),
FOREIGN KEY (status_id) REFERENCES lookup_order_status(status_id),
FOREIGN KEY (payment_mode_id) REFERENCES lookup_payment_mode(payment_mode_id),
FOREIGN KEY (channel_id) REFERENCES lookup_channel(channel_id)
);
```

sales_order_items

```
sql
CREATE TABLE sales_order_items (
order_item_id BIGINT PRIMARY KEY AUTO_INCREMENT,
order_id BIGINT NOT NULL,
product_id INT NOT NULL,
quantity INT NOT NULL,
```

unit_price DECIMAL(10,2) NOT NULL,

discount_percent DECIMAL(8,4) DEFAULT 0,

discount_amount DECIMAL(10,2) DEFAULT 0,

net_price DECIMAL(10,2) NOT NULL,

FOREIGN KEY (order_id) REFERENCES sales_orders(order_id) ON DELETE CASCADE,

FOREIGN KEY (product_id) REFERENCES products(product_id)

);

inventory

sql

CREATE TABLE inventory (

inventory_id BIGINT PRIMARY KEY AUTO_INCREMENT,

product_id INT NOT NULL,

store_id INT NOT NULL,

quantity_on_hand INT DEFAULT 0,

quantity_reserved INT DEFAULT 0,

quantity_available INT GENERATED ALWAYS AS (quantity_on_hand - quantity_reserved) STORED,

reorder_point INT DEFAULT 10,

last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

UNIQUE KEY uk_product_store (product_id, store_id),

FOREIGN KEY (product_id) REFERENCES products(product_id),

FOREIGN KEY (store_id) REFERENCES stores(store_id)

);

3.4 Entity Relationship Diagram

text

■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■

■ customers ■ ■ stores ■ ■ suppliers ■

■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■

■ customer_id ■■■■■■ ■ store_id ■■■■■■ ■ supplier_id ■

■ customer_code■ ■ ■ store_code ■ ■ ■ supplier_code■

■ name ■ ■ ■ name ■ ■ ■ name ■

■■■■■■■■■■■■■■■■■■ ■ ■■■■■■■■■■■■■■■■■■ ■ ■■■■■■■■■■■■■■■■■■

■ ■

▼ ▼

■■

■ sales_orders ■

■■

■ order_id (PK) ■

■ order_number (UNIQUE) ■

■ customer_id (FK) ■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ store_id (FK) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ order_date ■ ■

■ status_id (FK) ■ ■

■ payment_mode_id (FK) ■ ■

■■■ ■

■ ■

▼ ■

■■■ ■

■ sales_order_items ■ ■

■■■ ■

■ order_item_id (PK) ■ ■

■ order_id (FK) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ product_id (FK) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ quantity ■ ■

■ unit_price ■ ■

■ net_price ■ ■

■■■ ■

■ ■

▼ ■

■■■ ■

■ products ■ ■

■■■ ■

■ product_id (PK) ■ ■

■ product_code (UNIQUE) ■ ■

■ brand_id (FK) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ category_id (FK) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ sub_category_id (FK) ■ ■ ■

■ mrp ■ ■ ■

■ cost_price ■ ■ ■

■■■ ■ ■

■ ■ ■

▼ ■ ■

■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ ■ ■

■ brands ■ ■ categories ■ ■ ■

■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ ■ ■

■ brand_id ■■■■■■■ category_id ■ ■ ■

■ brand_name ■ ■ name ■ ■ ■

■■■■■■■■■■■■■■■■ ■ parent_id ■ ■ ■

■■■■■■■■■■■■■■■■ ■ ■

■ ■

■■■ ■ ■

■ product_suppliers ■ ■ ■

■■■ ■ ■

■ product_id (FK) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■

■ supplier_id (FK) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ is_primary ■

■■■

3.5 Normalization Level

Normal Form Status Explanation

1NF ■ Atomic values, no repeating groups

2NF ■ No partial dependencies

3NF ■ No transitive dependencies

BCNF ■ All determinants are candidate keys

4. Target System: retail_rolap

4.1 Overview

retail_rolap is the Relational OLAP system designed for analytical queries and business intelligence. It uses a Star Schema with denormalized dimensions for fast query performance.

4.2 Schema Structure

text

retail_rolap/

■■■ Dimensions (Denormalized)/

■ ■■■ dim_date -- Date dimension with fiscal calendar

■ ■■■ dim_product -- Product with category, brand denormalized

■ ■■■ dim_store -- Store with geography denormalized

■ ■■■ dim_customer -- Customer with segmentation

■ ■■■ dim_supplier -- Supplier information

■

■■■ Fact Table/

■ ■■■ fact_sales -- Sales transactions (grain: line item)

■

■■■ Aggregate Tables/

■■■ agg_daily_store_sales

■■■ agg_daily_product_sales

■■■ agg_monthly_category_sales

■■■ agg_monthly_store_performance

■■■ agg_customer_lifetime

4.3 Dimension Tables

dim_date

sql

```
CREATE TABLE dim_date (
date_id INT PRIMARY KEY, -- YYYYMMDD format
full_date DATE NOT NULL,
year_val SMALLINT,
quarter_val TINYINT,
quarter_label VARCHAR(20), -- Q1, Q2, Q3, Q4
month_val TINYINT,
month_name VARCHAR(20),
month_short VARCHAR(3),
day_val TINYINT,
day_of_week TINYINT, -- 1=Sunday, 7=Saturday
weekday_name VARCHAR(20),
weekday_short VARCHAR(3),
week_of_year TINYINT,
is_weekend BOOLEAN,
is_holiday BOOLEAN DEFAULT FALSE,
holiday_name VARCHAR(100),
fiscal_year SMALLINT, -- April-March fiscal year
fiscal_quarter TINYINT,
fiscal_month TINYINT
);
```

dim_product (Denormalized)

sql

```
CREATE TABLE dim_product (
product_key INT PRIMARY KEY AUTO_INCREMENT,
product_id VARCHAR(20) NOT NULL UNIQUE,
product_name VARCHAR(255),
brand VARCHAR(100), -- Denormalized from brands
category VARCHAR(100), -- Denormalized from categories
sub_category VARCHAR(100),
mrp DECIMAL(10,2),
cost_price DECIMAL(10,2),
```

gross_margin_pct DECIMAL(10,4),
is_perishable BOOLEAN,
veg_nonveg VARCHAR(20)
);
dim_store
sql
CREATE TABLE dim_store (
store_key INT PRIMARY KEY AUTO_INCREMENT,
store_id VARCHAR(20) NOT NULL UNIQUE,
store_name VARCHAR(255),
city VARCHAR(100),
state VARCHAR(100),
city_tier TINYINT,
cluster_zone VARCHAR(50),
store_type VARCHAR(50),
store_size_sqft INT,
opening_date DATE
);
dim_customer
sql
CREATE TABLE dim_customer (
customer_key INT PRIMARY KEY AUTO_INCREMENT,
customer_id VARCHAR(20) NOT NULL UNIQUE,
customer_name VARCHAR(255),
gender VARCHAR(20),
age_group VARCHAR(50),
city VARCHAR(100),
state VARCHAR(100),
city_tier TINYINT,
is_registered BOOLEAN,
loyalty_member BOOLEAN,
customer_segment VARCHAR(20) -- VIP, Premium, Regular, Occasional
);
dim_supplier
sql
CREATE TABLE dim_supplier (
supplier_key INT PRIMARY KEY AUTO_INCREMENT,
supplier_id VARCHAR(20) NOT NULL UNIQUE,

supplier_name VARCHAR(255),
supplier_type VARCHAR(100),
city VARCHAR(100),
state VARCHAR(100),
payment_days INT,
lead_time_days INT,
is_preferred_vendor BOOLEAN
);
4.4 Fact Table
fact_sales
sql
CREATE TABLE fact_sales (
sale_key BIGINT PRIMARY KEY AUTO_INCREMENT,
transaction_id VARCHAR(30),
date_id INT NOT NULL, -- FK to dim_date
product_key INT NOT NULL, -- FK to dim_product
store_key INT NOT NULL, -- FK to dim_store
customer_key INT, -- FK to dim_customer
supplier_key INT, -- FK to dim_supplier
quantity INT NOT NULL,
unit_price DECIMAL(10,2),
discount_amount DECIMAL(10,2),
sale_amount DECIMAL(12,2) NOT NULL,
cost_amount DECIMAL(12,2),
profit DECIMAL(12,2),
margin_pct DECIMAL(8,4),
payment_mode VARCHAR(50),
is_return BOOLEAN DEFAULT FALSE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
4.5 Aggregate Tables
agg_daily_store_sales
sql
CREATE TABLE agg_daily_store_sales (
date_id INT NOT NULL,
store_key INT NOT NULL,
total_sales DECIMAL(14,2) DEFAULT 0,
total_quantity INT DEFAULT 0,

transaction_count INT DEFAULT 0,
unique_customers INT DEFAULT 0,
avg_ticket_size DECIMAL(10,2) DEFAULT 0,
PRIMARY KEY (date_id, store_key)
);
agg_monthly_category_sales
sql
CREATE TABLE agg_monthly_category_sales (
year_month INT NOT NULL, -- YYYYMM format
category VARCHAR(100) NOT NULL,
total_sales DECIMAL(14,2) DEFAULT 0,
total_profit DECIMAL(14,2) DEFAULT 0,
total_quantity INT DEFAULT 0,
transaction_count INT DEFAULT 0,
sales_rank INT DEFAULT 0,
PRIMARY KEY (year_month, category)
);
agg_customer_lifetime
sql
CREATE TABLE agg_customer_lifetime (
customer_key INT PRIMARY KEY,
first_purchase_date DATE,
last_purchase_date DATE,
total_spend DECIMAL(14,2) DEFAULT 0,
total_transactions INT DEFAULT 0,
avg_order_value DECIMAL(10,2) DEFAULT 0,
days_since_last_purchase INT DEFAULT 0,
lifetime_segment VARCHAR(20) -- Platinum, Gold, Silver, Bronze
);
4.6 Star Schema Diagram
text
■■■■■■■■■■■■■■■■■■■■
■ dim_date ■
■ - date_id (PK) ■
■ - full_date ■
■ - year_val ■
■ - month_name ■
■■■■■■■■■■■■■■■■■■■■

■

■■■■■■■■■■■■■■■■■■■■ ■ ■■■■■■■■■■■■■■■■■■■■

■ dim_product ■ ■ ■ dim_store ■

■ - product_key(PK)■■■■■■■■■■■■■■■■■■■■■■■■ - store_key(PK) ■

■ - product_name ■ ■ ■ - store_name ■

■ - brand ■ ■ ■ - city ■

■ - category ■ ▼ ■ - state ■

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■

■ fact_sales ■

■ - sale_key ■

■ - date_id(FK)■

■ - product(FK)■

■ - store(FK) ■

■ - customer(FK)■

■ - quantity ■

■ - sale_amount■

■■■■■■■■■■■■■■■■

■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ ■ ■

▼ ▼ ▼

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■

■ dim_customer ■ ■ dim_supplier ■ ■ ■

■ - customer_key ■ ■ - supplier_key ■ ■ ■

■ - customer_name ■ ■ - supplier_name ■ ■ ■

■ - segment ■ ■ - type ■ ■ ■

■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■

5. Target System: retail_molap

5.1 Overview

retail_molap is the Multidimensional OLAP system containing pre-aggregated cube structures for lightning-fast analytical queries. These cubes are designed for OLAP operations like Slice, Dice, Roll-up, and Drill-down.

5.2 Cube Structures

text

retail_molap/

■■■ Cube 1: Sales by Store x Product x Date (Detailed)

■■■ Cube 2: Sales by Category x Month (Rollup)

■■■ Cube 3: Sales by Customer Segment x Store Type (Cross-tab)

■■■ Cube 4: Time Series with Moving Averages

■■■ Cube 5: Product Hierarchy Rollups

■■■ Cube 6: Store Geography Hierarchy

■■■ Cube 7: Customer Segmentation Cube

■■■ Metadata Tables/

■■■ cube_metadata

■■■ dimension_metadata

■■■ measure_metadata

5.3 Cube Definitions

Cube 1: Sales by Store x Product x Date

sql

```
CREATE TABLE cube_sales_store_product_date (
cube_key BIGINT PRIMARY KEY AUTO_INCREMENT,
store_id VARCHAR(20),
product_id VARCHAR(20),
date_id INT,
year_val INT,
quarter_val INT,
month_val INT,
store_city VARCHAR(100),
store_state VARCHAR(100),
product_category VARCHAR(100),
product_brand VARCHAR(100),
total_sales DECIMAL(14,2),
total_quantity INT,
transaction_count INT,
unique_customers INT,
total_profit DECIMAL(14,2),
avg_margin_pct DECIMAL(8,4)
);
```

Use Case: Detailed sales analysis by any combination of store, product, and time.

Cube 2: Sales by Category x Month

sql

```
CREATE TABLE cube_sales_category_month (
cube_key BIGINT PRIMARY KEY AUTO_INCREMENT,
product_category VARCHAR(100),
year_month INT, -- YYYYMM format
year_val INT,
```

month_val INT,
total_sales DECIMAL(14,2),
total_quantity INT,
transaction_count INT,
total_profit DECIMAL(14,2),
prev_month_sales DECIMAL(14,2),
growth_pct DECIMAL(8,4)
);
Use Case: Monthly category performance with growth calculation.

Cube 3: Sales by Customer Segment x Store Type
sql
CREATE TABLE cube_sales_segment_storetype (
cube_key BIGINT PRIMARY KEY AUTO_INCREMENT,
customer_segment VARCHAR(20),
store_type VARCHAR(50),
year_val INT,
total_sales DECIMAL(14,2),
transaction_count INT,
unique_customers INT,
avg_order_value DECIMAL(10,2)
);
Use Case: Cross-tab analysis of customer segments vs store types.

Cube 4: Time Series with Moving Averages
sql
CREATE TABLE cube_time_series (
cube_key BIGINT PRIMARY KEY AUTO_INCREMENT,
time_level VARCHAR(20), -- DAY, WEEK, MONTH, QUARTER, YEAR
time_value VARCHAR(50),
time_label VARCHAR(100),
start_date DATE,
end_date DATE,
total_sales DECIMAL(14,2),
total_quantity INT,
transaction_count INT,
unique_customers INT,
moving_avg_7d DECIMAL(14,2),
moving_avg_30d DECIMAL(14,2)

);

Use Case: Trend analysis with moving averages for forecasting.

Cube 5: Product Hierarchy Rollups

sql

```
CREATE TABLE cube_product_hierarchy (
hierarchy_key BIGINT PRIMARY KEY AUTO_INCREMENT,
level_type VARCHAR(20), -- PRODUCT, BRAND, CATEGORY, ALL
level_value VARCHAR(100),
level_name VARCHAR(255),
parent_value VARCHAR(100),
total_sales DECIMAL(14,2),
total_quantity INT,
total_profit DECIMAL(14,2),
sales_share_pct DECIMAL(8,4)
);
```

Use Case: Drill-down/roll-up across product hierarchy.

Sample Data:

level_type level_name parent_value total_sales sales_share_pct
ALL All Products NULL 1,000,000 100.00
CATEGORY Electronics ALL 600,000 60.00
CATEGORY Clothing ALL 400,000 40.00
BRAND Apple Electronics 350,000 35.00
BRAND Samsung Electronics 250,000 25.00
PRODUCT iPhone 14 Apple 200,000 20.00

Cube 6: Store Geography Hierarchy

sql

```
CREATE TABLE cube_store_hierarchy (
hierarchy_key BIGINT PRIMARY KEY AUTO_INCREMENT,
level_type VARCHAR(20), -- STORE, CITY, STATE, CLUSTER, ALL
level_value VARCHAR(100),
level_name VARCHAR(255),
parent_value VARCHAR(100),
total_sales DECIMAL(14,2),
transaction_count INT,
unique_customers INT,
avg_ticket_size DECIMAL(10,2)
);
```

Use Case: Geographic drill-down analysis.

Cube 7: Customer Segmentation Cube

sql

```sql
CREATE TABLE cube_customer_segmentation (
cube_key BIGINT PRIMARY KEY AUTO_INCREMENT,
customer_segment VARCHAR(20),
age_group VARCHAR(50),
city_tier TINYINT,
loyalty_member BOOLEAN,
customer_count INT,
total_sales DECIMAL(14,2),
avg_order_value DECIMAL(10,2),
avg_transactions_per_customer DECIMAL(8,2)
);
```

Use Case: Customer behavior analysis by segment.

5.4 OLAP Operations Supported

Operation Description Supported Cubes SQL Example

Slice Select a single dimension value All cubes WHERE category = 'Electronics'

Dice Select a sub-cube (multiple dimensions) Cubes 1, 2, 3 WHERE year_val = 2025 AND category = 'Electronics'

Drill-down Navigate from summary to detail Cubes 5, 6 From CATEGORY to BRAND to PRODUCT

Roll-up Aggregate from detail to summary Cubes 5, 6 From PRODUCT to BRAND to CATEGORY

Pivot Rotate dimensions for cross-tabulation Cube 3 Customer segments as rows, store types as columns

Time Series Analyze trends over time Cube 4 Daily sales with 7-day and 30-day moving averages

6. ETL Pipeline

6.1 ETL Architecture

text

■■

■ ETL PIPELINE ■

■■

■ ■

■ Phase 1: Load OLTP from DW (ONE-TIME) ■

■ ■■ ■

■ ■ retail_dw ■■■ retail_oltp ■ ■

■ ■ • Categories ← dim_product.category ■ ■

■ ■ • Brands ← dim_product.brand ■ ■

■ ■ • Products ← dim_product ■ ■

■ ■ • Customers ← dim_customer ■ ■

■ ■ • Stores ← dim_store ■ ■

■ ■ • Sales Orders ← fact_transaction (aggregated) ■ ■

■ ■■
■■■■■■■■■■■■■■■ ■

■ ■ ■

■ ▼ ■

■ Phase 2: Load ROLAP from OLTP (Incremental) ■

■ ■■
■■■■■■■■■■■■■■■ ■

■ ■ retail_oltp ■■■ retail_rolap ■ ■

■ ■ • dim_date ← Date generation ■ ■

■ ■ • dim_product ← products + brands + categories ■ ■

■ ■ • dim_store ← stores ■ ■

■ ■ • dim_customer ← customers + lookup_gender ■ ■

■ ■ • fact_sales ← sales_orders + sales_order_items ■ ■

■ ■ • Aggregates ← fact_sales aggregations ■ ■

■ ■■
■■■■■■■■■■■■■■■ ■

■ ■ ■

■ ▼ ■

■ Phase 3: Load MOLAP from ROLAP (Incremental) ■

■ ■■
■■■■■■■■■■■■■■■ ■

■ ■ retail_rolap ■■■ retail_molap ■ ■

■ ■ • Cube 1 ← fact_sales + dimensions ■ ■

■ ■ • Cube 2 ← agg_monthly_category_sales ■ ■

■ ■ • Cube 3 ← fact_sales + dimensions ■ ■

■ ■ • Cube 4 ← fact_sales aggregation ■ ■

■ ■ • Cube 5 ← fact_sales + product hierarchy ■ ■

■ ■ • Cube 6 ← fact_sales + store hierarchy ■ ■

■ ■ • Cube 7 ← dim_customer + fact_sales ■ ■

■ ■■
■■■■■■■■■■■■■■■ ■

■ ■

■■■
■■■■■■■■■■■■■■■■■■■■■■

6.2 ETL Procedures

Phase 1: One-Time Load from retail_dw to retail_oltp

Procedure: sp_one_time_load_from_retail_dw()

sql

Steps:

1. LOAD categories FROM dim_product (category, sub_category)

2. LOAD brands FROM dim_product (brand)

3. LOAD suppliers FROM dim_supplier

4. LOAD stores FROM dim_store

5. LOAD customers FROM dim_customer

6. LOAD products FROM dim_product (with FK mapping)

7. LOAD product_suppliers (cross-reference)

8. LOAD sales_orders FROM fact_transaction (aggregated by transaction)

9. LOAD sales_order_items FROM fact_transaction

10. LOAD inventory FROM fact_inventory (latest snapshot)

Phase 2: ROLAP ETL

Procedure: sp_run_rolap_etl()

sql

Steps:

1. CALL sp_populate_date_dimension(2020, 2026) -- Populate date dimension

2. CALL sp_load_rolap_dimensions() -- Load all dimensions

3. CALL sp_load_fact_sales() -- Load fact table

4. CALL sp_refresh_aggregates() -- Refresh aggregates

Dimension Loading (sp_load_rolap_dimensions):

sql

- dim_product: SELECT FROM retail_oltp.products

JOIN brands, categories (denormalize)

- dim_store: SELECT FROM retail_oltp.stores

- dim_customer: SELECT FROM retail_oltp.customers

JOIN lookup_gender, calculate segment

- dim_supplier: SELECT FROM retail_oltp.suppliers

Fact Loading (sp_load_fact_sales):

sql

- JOIN sales_orders + sales_order_items

- Map to dimension keys

- Calculate profit = net_price - (quantity * cost_price)

- Calculate margin_pct = (profit / net_price) * 100

- Filter status_id = 3 (Completed orders only)

Aggregate Refresh (sp_refresh_aggregates):

sql

- agg_daily_store_sales: GROUP BY date_id, store_key
- agg_daily_product_sales: GROUP BY date_id, product_key
- agg_monthly_category_sales: GROUP BY year_month, category
- agg_monthly_store_performance: GROUP BY year_month, store_key
- agg_customer_lifetime: Customer aggregates with segmentation

Phase 3: MOLAP Cube Refresh

Procedure: sp_refresh_all_molap_cubes()

sql

Steps:

1. CALL sp_refresh_cube_sales_store_product_date()
2. CALL sp_refresh_cube_sales_category_month()
3. CALL sp_refresh_cube_sales_segment_storetype()
4. CALL sp_refresh_cube_time_series()
5. CALL sp_refresh_cube_product_hierarchy()
6. CALL sp_refresh_cube_store_hierarchy()
7. CALL sp_refresh_cube_customer_segmentation()

6.3 ETL Control Tables

sql

```
-- Track ETL batches
CREATE TABLE etl_batch_log (
batch_id INT PRIMARY KEY AUTO_INCREMENT,
batch_start_time DATETIME NOT NULL,
batch_end_time DATETIME,
records_processed INT DEFAULT 0,
status VARCHAR(20),
error_message TEXT
);
```

```
-- Track ETL control (for incremental loads)
CREATE TABLE etl_control (
control_id INT PRIMARY KEY AUTO_INCREMENT,
etl_name VARCHAR(100) NOT NULL,
last_batch_date DATETIME,
last_processed_order_id BIGINT,
status VARCHAR(20)
);
```

6.4 Data Validation Queries

sql

```
-- Verify record counts across systems
SELECT 'Source (retail_dw)' AS System, COUNT(*) AS Customers FROM retail_dw.dim_customer
UNION ALL
SELECT 'OLTP', COUNT(*) FROM retail_oltp.customers
UNION ALL
SELECT 'ROLAP', COUNT(*) FROM retail_rolap.dim_customer
UNION ALL
SELECT 'MOLAP (unique customers)', COUNT(DISTINCT customer_segment) FROM
retail_molap.cube_customer_segmentation;
```

7. Analytical Queries

7.1 ROLAP Queries

Query 1: Daily Sales Trend (Last 30 days)

sql

```
SELECT
d.full_date,
d.weekday_name,
COALESCE(SUM(fs.sale_amount), 0) AS daily_sales,
COUNT(DISTINCT fs.transaction_id) AS orders,
COUNT(DISTINCT fs.customer_key) AS unique_customers,
ROUND(COALESCE(SUM(fs.sale_amount), 0) / NULLIF(COUNT(DISTINCT fs.transaction_id), 0),
2) AS avg_order_value
FROM dim_date d
LEFT JOIN fact_sales fs ON d.date_id = fs.date_id
WHERE d.full_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
AND d.full_date <= CURDATE()
GROUP BY d.date_id, d.full_date, d.weekday_name
ORDER BY d.full_date DESC;
```

Query 2: Top Categories by Sales

sql

```
SELECT
dp.category,
SUM(fs.sale_amount) AS total_sales,
SUM(fs.quantity) AS units_sold,
COUNT(DISTINCT fs.transaction_id) AS orders,
ROUND(AVG(fs.margin_pct), 2) AS avg_margin_pct,
ROUND(SUM(fs.profit), 2) AS total_profit
FROM fact_sales fs
```

```
JOIN dim_product dp ON fs.product_key = dp.product_key
GROUP BY dp.category
ORDER BY total_sales DESC
LIMIT 10;
```

Query 3: Store Performance Dashboard

```
sql
SELECT
ds.store_name,
ds.city,
ds.store_type,
SUM(fs.sale_amount) AS total_sales,
COUNT(DISTINCT fs.transaction_id) AS order_count,
COUNT(DISTINCT fs.customer_key) AS unique_customers,
ROUND(AVG(fs.sale_amount), 2) AS avg_ticket_size,
ROUND(SUM(fs.profit), 2) AS total_profit
FROM fact_sales fs
JOIN dim_store ds ON fs.store_key = ds.store_key
GROUP BY ds.store_key, ds.store_name, ds.city, ds.store_type
ORDER BY total_sales DESC;
```

Query 4: Customer Segmentation Analysis

```
sql
SELECT
dc.customer_segment,
COUNT(DISTINCT dc.customer_key) AS customer_count,
SUM(fs.sale_amount) AS total_sales,
COUNT(DISTINCT fs.transaction_id) AS total_orders,
ROUND(SUM(fs.sale_amount) / NULLIF(COUNT(DISTINCT dc.customer_key), 0), 2) AS
avg_spend_per_customer,
ROUND(AVG(fs.sale_amount), 2) AS avg_order_value
FROM dim_customer dc
LEFT JOIN fact_sales fs ON dc.customer_key = fs.customer_key
GROUP BY dc.customer_segment
ORDER BY total_sales DESC;
```

Query 5: Monthly Category Performance with Ranking

```
sql
SELECT
year_month,
category,
```

```
total_sales,
total_quantity,
sales_rank,
ROUND((total_profit / NULLIF(total_sales, 0)) * 100, 2) AS margin_pct
FROM agg_monthly_category_sales
WHERE year_month >= YEAR(CURDATE()) * 100 + 1
ORDER BY year_month DESC, sales_rank;
```

Query 6: Customer Lifetime Value Analysis

```
sql
SELECT
lifetime_segment,
COUNT(*) AS customer_count,
ROUND(AVG(total_spend), 2) AS avg_lifetime_value,
ROUND(AVG(total_transactions), 1) AS avg_transactions,
ROUND(AVG(avg_order_value), 2) AS avg_order_value,
ROUND(AVG(days_since_last_purchase), 1) AS avg_days_inactive
FROM agg_customer_lifetime
GROUP BY lifetime_segment
ORDER BY avg_lifetime_value DESC;
```

Query 7: Best Selling Products (Last 90 days)

```
sql
SELECT
dp.product_name,
dp.brand,
dp.category,
SUM(fs.sale_amount) AS revenue,
SUM(fs.quantity) AS units_sold,
COUNT(DISTINCT fs.transaction_id) AS order_count,
ROUND(AVG(fs.margin_pct), 2) AS avg_margin
FROM fact_sales fs
JOIN dim_product dp ON fs.product_key = dp.product_key
WHERE fs.date_id >= DATE_FORMAT(DATE_SUB(CURDATE(), INTERVAL 90 DAY),
'%Y%m%d')
GROUP BY dp.product_key, dp.product_name, dp.brand, dp.category
ORDER BY revenue DESC
LIMIT 20;
```

Query 8: Hourly Sales Pattern

```
sql
```

```
SELECT
HOUR(so.order_date) AS hour_of_day,
COUNT(*) AS order_count,
ROUND(AVG(so.net_amount), 2) AS avg_order_value,
SUM(so.net_amount) AS total_sales
FROM retail_oltp.sales_orders so
WHERE so.order_date >= DATE_SUB(NOW(), INTERVAL 30 DAY)
AND so.status_id = 3
GROUP BY HOUR(so.order_date)
ORDER BY hour_of_day;
```

7.2 MOLAP Queries

Query: Product Hierarchy Drill-Down

sql

```
SELECT
level_type,
level_name,
parent_value,
ROUND(total_sales, 2) AS total_sales,
ROUND(sales_share_pct, 2) AS market_share
FROM cube_product_hierarchy
WHERE level_type IN ('CATEGORY', 'PRODUCT')
ORDER BY FIELD(level_type, 'CATEGORY', 'PRODUCT'), total_sales DESC;
```

Query: Store Geography Rollup

sql

```
SELECT
level_type,
level_name,
parent_value,
ROUND(total_sales, 2) AS total_sales,
transaction_count,
unique_customers,
ROUND(avg_ticket_size, 2) AS avg_ticket_size
FROM cube_store_hierarchy
ORDER BY FIELD(level_type, 'ALL', 'CLUSTER', 'STATE', 'CITY', 'STORE'), total_sales DESC;
```

Query: Time Series with Moving Averages

sql

```
SELECT
time_label,
```

```
ROUND(total_sales, 2) AS sales,
ROUND(moving_avg_7d, 2) AS ma_7d,
ROUND(moving_avg_30d, 2) AS ma_30d
FROM cube_time_series
WHERE time_level = 'DAY'
AND start_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
ORDER BY start_date DESC;
```

Query: Pivot - Monthly Sales by Category

sql

```
SELECT
product_category,
MAX(CASE WHEN month_val = 1 THEN total_sales ELSE 0 END) AS Jan,
MAX(CASE WHEN month_val = 2 THEN total_sales ELSE 0 END) AS Feb,
MAX(CASE WHEN month_val = 3 THEN total_sales ELSE 0 END) AS Mar,
MAX(CASE WHEN month_val = 4 THEN total_sales ELSE 0 END) AS Apr,
MAX(CASE WHEN month_val = 5 THEN total_sales ELSE 0 END) AS May,
MAX(CASE WHEN month_val = 6 THEN total_sales ELSE 0 END) AS Jun,
MAX(CASE WHEN month_val = 7 THEN total_sales ELSE 0 END) AS Jul,
MAX(CASE WHEN month_val = 8 THEN total_sales ELSE 0 END) AS Aug,
MAX(CASE WHEN month_val = 9 THEN total_sales ELSE 0 END) AS Sep,
MAX(CASE WHEN month_val = 10 THEN total_sales ELSE 0 END) AS Oct,
MAX(CASE WHEN month_val = 11 THEN total_sales ELSE 0 END) AS Nov,
MAX(CASE when month_val = 12 THEN total_sales ELSE 0 END) AS `Dec`,
SUM(total_sales) AS Total
FROM cube_sales_category_month
WHERE year_val = 2025
GROUP BY product_category WITH ROLLUP;
```

Query: Customer Segment vs Store Type Cross-tab

sql

```
SELECT
customer_segment,
MAX(CASE WHEN store_type = 'Hypermarket' THEN total_sales ELSE 0 END) AS Hypermarket,
MAX(CASE WHEN store_type = 'Supermarket' THEN total_sales ELSE 0 END) AS Supermarket,
MAX(CASE WHEN store_type = 'Express' THEN total_sales ELSE 0 END) AS Express,
SUM(total_sales) AS Total
FROM cube_sales_segment_storetype
WHERE year_val = 2025
GROUP BY customer_segment
```

ORDER BY Total DESC;

8. Execution Guide

8.1 Prerequisites

MySQL Server 8.0.41 or higher

Existing retail_dw database with data

Sufficient disk space (minimum 5GB)

MySQL user with CREATE, DROP, INSERT, SELECT privileges

8.2 File Structure

text

retail_analytics_project/

■

■■■ 01_create_oltp_from_dw.sql

■■■ 02_one_time_load_from_dw.sql

■■■ 03_verify_load_and_backup.sql

■■■ 04_deprecate_retail_dw.sql (optional)

■

■■■ 01_create_rolap_schema.sql

■■■ 02_create_molap_schema.sql

■■■ 03_etl_rolap_from_oltp.sql

■■■ 04_etl_molap_from_rolap.sql

■■■ 05_analytical_queries_rolap.sql

■■■ 06_analytical_queries_molap.sql

■

■■■ run_all.sql (master script)

8.3 Execution Order

bash

Step 1: Create OLTP database and load from retail_dw

mysql -u root -p < 01_create_oltp_from_dw.sql

mysql -u root -p < 02_one_time_load_from_dw.sql

mysql -u root -p < 03_verify_load_and_backup.sql

Step 2: Create ROLAP database and load from OLTP

mysql -u root -p < 01_create_rolap_schema.sql

mysql -u root -p < 03_etl_rolap_from_oltp.sql

Step 3: Create MOLAP database and load from ROLAP

mysql -u root -p < 02_create_molap_schema.sql

mysql -u root -p < 04_etl_molap_from_rolap.sql

```
# Step 4: Run analytical queries (optional)
mysql -u root -p < 05_analytical_queries_rolap.sql
mysql -u root -p < 06_analytical_queries_molap.sql

# Step 5: Deprecate retail_dw (after verification)
mysql -u root -p < 04_deprecate_retail_dw.sql
```

8.4 Master Execution Script

Create run_all.sql:

```
sql
-- ============================================
-- MASTER EXECUTION SCRIPT
-- Run complete migration in correct order
-- ============================================

SET FOREIGN_KEY_CHECKS = 0;

SELECT '========================================' AS '';
SELECT 'STARTING COMPLETE MIGRATION' AS '';
SELECT '========================================' AS '';

-- Phase 1: OLTP Setup
SELECT 'Phase 1: Setting up OLTP...' AS '';
SOURCE 01_create_oltp_from_dw.sql;
SOURCE 02_one_time_load_from_dw.sql;
SOURCE 03_verify_load_and_backup.sql;

-- Phase 2: ROLAP Setup
SELECT 'Phase 2: Setting up ROLAP...' AS '';
SOURCE 01_create_rolap_schema.sql;
SOURCE 03_etl_rolap_from_oltp.sql;

-- Phase 3: MOLAP Setup
SELECT 'Phase 3: Setting up MOLAP...' AS '';
SOURCE 02_create_molap_schema.sql;
SOURCE 04_etl_molap_from_rolap.sql;

SET FOREIGN_KEY_CHECKS = 1;

SELECT '========================================' AS '';
SELECT 'MIGRATION COMPLETED SUCCESSFULLY' AS '';
SELECT '========================================' AS '';
```

8.5 Verification Queries

After execution, run these verification queries:

sql

```
-- Check record counts across all systems
SELECT '=== SYSTEM VERIFICATION ===' AS '';

SELECT 'OLTP' AS System, 'Customers' AS TableName, COUNT(*) AS Count FROM retail_oltp.customers
UNION ALL SELECT 'OLTP', 'Products', COUNT(*) FROM retail_oltp.products
UNION ALL SELECT 'OLTP', 'Stores', COUNT(*) FROM retail_oltp.stores
UNION ALL SELECT 'OLTP', 'Sales Orders', COUNT(*) FROM retail_oltp.sales_orders
UNION ALL SELECT 'ROLAP', 'dim_date', COUNT(*) FROM retail_rolap.dim_date
UNION ALL SELECT 'ROLAP', 'dim_product', COUNT(*) FROM retail_rolap.dim_product
UNION ALL SELECT 'ROLAP', 'fact_sales', COUNT(*) FROM retail_rolap.fact_sales
UNION ALL SELECT 'MOLAP', 'cube_sales_store_product_date', COUNT(*) FROM retail_molap.cube_sales_store_product_date
UNION ALL SELECT 'MOLAP', 'cube_product_hierarchy', COUNT(*) FROM retail_molap.cube_product_hierarchy
UNION ALL SELECT 'MOLAP', 'cube_store_hierarchy', COUNT(*) FROM retail_molap.cube_store_hierarchy;
```

9. Troubleshooting

9.1 Common Errors and Solutions

Error Code Error Message Cause Solution

1064 Syntax error near 'current_date' Using reserved keyword as variable Rename variable to v_current_date

1093 Can't specify target table for update in FROM clause Self-join in UPDATE statement Use temporary table or derived table

1054 Unknown column 'year_month' Missing backticks around reserved word Add backticks: `year_month`

1146 Table 'retail_oltp.products' doesn't exist OLTP not created yet Run OLTP creation scripts first

1452 Cannot add foreign key constraint Referenced table empty or missing Load dimension tables before fact tables

1265 Data truncated for column Data type mismatch Check column definitions vs source data

9.2 Debugging Queries

sql

```
-- Check if tables exist
SELECT table_schema, table_name
FROM information_schema.tables
WHERE table_schema IN ('retail_oltp', 'retail_rolap', 'retail_molap')
ORDER BY table_schema, table_name;

-- Check stored procedures
SELECT db, name, type
FROM mysql.proc
```

```
WHERE db IN ('retail_oltp', 'retail_rolap', 'retail_molap');

-- Check foreign key issues
SELECT * FROM information_schema.KEY_COLUMN_USAGE
WHERE constraint_schema IN ('retail_oltp', 'retail_rolap', 'retail_molap')
AND referenced_table_name IS NOT NULL;

-- Check data distribution by date
SELECT
d.year_val,
d.month_name,
COUNT(DISTINCT fs.transaction_id) AS transactions,
SUM(fs.sale_amount) AS total_sales
FROM retail_rolap.fact_sales fs
JOIN retail_rolap.dim_date d ON fs.date_id = d.date_id
GROUP BY d.year_val, d.month_val, d.month_name
ORDER BY d.year_val DESC, d.month_val DESC;
```

9.3 Performance Optimization

```
sql
-- Add indexes for better query performance
USE retail_rolap;

-- Indexes for fact_sales
CREATE INDEX idx_fact_date ON fact_sales(date_id);
CREATE INDEX idx_fact_product ON fact_sales(product_key);
CREATE INDEX idx_fact_store ON fact_sales(store_key);
CREATE INDEX idx_fact_customer ON fact_sales(customer_key);

-- Indexes for dimensions
CREATE INDEX idx_product_category ON dim_product(category);
CREATE INDEX idx_product_brand ON dim_product(brand);
CREATE INDEX idx_store_city ON dim_store(city);
CREATE INDEX idx_customer_segment ON dim_customer(customer_segment);

-- Analyze tables for query optimizer
ANALYZE TABLE fact_sales, dim_product, dim_store, dim_customer, dim_date;
```

9.4 Data Quality Checks

```
sql
-- Check for orphaned records
SELECT 'Orders without customers' AS Issue, COUNT(*) AS Count
FROM retail_oltp.sales_orders so
```

```
LEFT JOIN retail_oltp.customers c ON so.customer_id = c.customer_id
WHERE so.customer_id IS NOT NULL AND c.customer_id IS NULL

UNION ALL

SELECT 'Order items without products', COUNT(*)
FROM retail_oltp.sales_order_items soi
LEFT JOIN retail_oltp.products p ON soi.product_id = p.product_id
WHERE soi.product_id IS NOT NULL AND p.product_id IS NULL

UNION ALL

SELECT 'Products without categories', COUNT(*)
FROM retail_oltp.products p
LEFT JOIN retail_oltp.categories c ON p.category_id = c.category_id
WHERE p.category_id IS NOT NULL AND c.category_id IS NULL;
```

10. Appendix

10.1 File Descriptions

File Name	Purpose	Dependencies
01_create_oltp_from_dw.sql	Creates OLTP schema	None
02_one_time_load_from_dw.sql	Loads data from retail_dw to OLTP	retail_dw, OLTP schema
03_verify_load_and_backup.sql	Validates and creates backup	OLTP populated
04_deprecate_retail_dw.sql	Marks retail_dw as deprecated	Verification passed
01_create_rolap_schema.sql	Creates ROLAP star schema	None
02_create_molap_schema.sql	Creates MOLAP cube structures	None
03_etl_rolap_from_oltp.sql	Loads ROLAP from OLTP	OLTP, ROLAP schema
04_etl_molap_from_rolap.sql	Loads MOLAP from ROLAP	ROLAP, MOLAP schema
05_analytical_queries_rolap.sql	Sample ROLAP queries	ROLAP populated
06_analytical_queries_molap.sql	Sample MOLAP queries	MOLAP populated

10.2 System Requirements

Resource	Minimum	Recommended
CPU	2 cores	4+ cores
RAM	4 GB	8+ GB
Disk Space	5 GB	20+ GB
MySQL Version	8.0	8.0.41+

10.3 Glossary

Term	Definition
OLTP	Online Transaction Processing - optimized for write operations
ROLAP	Relational OLAP - star schema for analytical queries
MOLAP	Multidimensional OLAP - pre-aggregated cubes
ETL	Extract, Transform, Load - data pipeline process

Star Schema Denormalized dimensions around a central fact table

3NF Third Normal Form - normalized schema design

Grain The level of detail represented in a fact table

Dimension Descriptive attributes for filtering and grouping

Fact Table Quantitative metrics for analysis

Cube Multi-dimensional data structure for OLAP

10.4 Contact & Support

For issues or questions regarding this migration:

Documentation Version: 1.0

Last Updated: May 30, 2026

MySQL Version: 8.0.41

Support: Refer to troubleshooting section above

Conclusion

This documentation provides a complete guide for migrating from the existing retail_dw data warehouse to a modern three-tier architecture comprising:

retail_oltp - Transaction processing system (3NF)

retail_rolap - Relational analytical system (Star Schema)

retail_molap - Multidimensional analytical system (Cubes)

The migration is designed as a one-time load from retail_dw to retail_oltp, after which retail_dw is deprecated. All subsequent ETL processes flow from OLTP $\rightarrow$ ROLAP $\rightarrow$ MOLAP, enabling comprehensive business intelligence capabilities.

End of Documentation